

1 配置一个专门符合 zypher 开发的环境

linux 基金会子项目，免费开源，符合规范上传。

linux 安装方法：

```
使用 zephyr_ubuntu22_04_install.sh 脚本，拷贝到自己 ubuntu 的家目录下，  
使用 cd ~ 命令进入家目录，然后 sudo chmod 777 zephyr_ubuntu22_04_install.sh  
赋予执行权限，最后 ./zephyr_ubuntu22_04_install.sh 运行会自动安装 zephyr 的环境。
```

开发环境搭建

完成 zephyr_ubuntu22_04_install.sh 下载之后，readme 有下载调试说明。

2 驱动移植

模块负责人只需要看驱动移植这一章节即可，以 I2C 驱动移植为例子：

1. 驱动核心（drivers/i2c/）---》模块负责人必须要完成

drivers/i2c/i2c_n32.c	I2C 驱动编写
drivers/i2c/Kconfig.n32	新建 I2C_N32 选项，select USE_NSING_I2C
drivers/i2c/CMakeLists.txt	加一行 zephyr_library_sources_ifdef(CONFIG_I2C_N32 i2c_n32.c)
drivers/i2c/Kconfig	加一行 source "drivers/i2c/Kconfig.n32"

2. 设备树（Devicetree）---》需要模块负责人检查

zephyr/dts/arm/nsing/n32g45x/n32g45x.dtsi 整个 MCU 相关外设的描述
各模块负责人到目录检查自己的外设，我编写了外设设备树(注释掉了需要编写后开启)，
但是还是要各模块负责人再自己确认。而且各外设参数信息都是通过设备树来传递到自己
编写的驱动里面的，通过下面的 nsing,n32-i2c.yaml 文件来增加或删除。

```
i2c1: i2c@40005400 {
    // 通过 DT_DRV_COMPAT = nsing_n32_i2c 与 设备树绑定。
    compatible = "nsing,n32-i2c";
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <0x40005400 0x400>; // 基地址和寄存器个数
    clocks = <&rcc N32_CLOCK_I2C1>; // 外设时钟
    interrupts = <31 0>, <32 0>; // 中断号，优先级
    interrupt-names = "event", "error"; // 事件，错误中断
    status = "disabled"; // 是否使用 I2C 外设，MCU 芯片外设树一般为 disabled
};
```

zephyr/boards/nsing/n32g45xml_stb/n32g45xml_stb.dts

模块负责人需要自己按照板子外设情况适配添加

```
&i2c1
{
    // 引脚属性
    pinctrl-0 = <&i2c1_scl_pb6 &i2c1_sda_pb7>;
    // 外设名称描述
    pinctrl-names = "default";
    // I2C 速率
    clock-frequency = < I2C_BITRATE_STANDARD >;
    status = "okay";
};
```

zephyr/boards/nsing/n32g45xml_stb/n32g45xml_stb-pinctrl.dtsi

// 使用 Pinctrl 系统自己添加引脚复用关系

```
i2c1_scl_pb6: i2c1_scl_pb6 {
```

```
pinmux = <N32_PINMUX(N32_PORT_B, 6, AF_OD, SLEW_RATE_2MHZ)>;

};

i2c1_sda_pb7: i2c1_sda_pb7 {
    pinmux = <N32_PINMUX(N32_PORT_B, 7, AF_OD, SLEW_RATE_2MHZ)>;

};
```

3. 设备树绑定 (Bindings / yaml) ---》 模块负责人需要自己通过相应驱动的yaml文件来增加或者删除设备树参数，每个模块都要编写

```
zephyr/dts/bindings/i2c/nsing,n32-i2c.yaml
zephyr/dts/bindings/i2c/nsing,n32g45-pinctrl.yaml (这个文件无需配置)
```

4. 引脚复用 (pinctrl) ---》 模块负责人不需要管

```
zephyr/drivers/pinctrl/pinctrl_n32.c
zephyr/drivers/pinctrl/Kconfig.n32
zephyr/include/zephyr/dt-bindings/pinctrl/n32-pinctrl.h
zephyr/soc/nsing/nsing_n32/common/pinctrl_soc.h
```

5. 时钟 (clock control) ---》 模块负责人不需要管

```
zephyr/drivers/clock_control/clock_control_n32.c
zephyr/drivers/clock_control/Kconfig.n32
zephyr/dts/bindings/clock/nsing,n32-rcc.yaml
zephyr/dts/bindings/clock/nsing,n32-pll-clock.yaml
```

6. 板级配置 (board) ---》 模块负责人需要勾选相应的模块使能

```
zephyr\boards\nsing\n32g45xml_stb\n32g45xml_stb_defconfig
选择外设配置 CONFIG_I2C=y
zephyr\boards\nsing\n32g45xml_stb\n32g45xml_stb.yaml
i2c 是支持的特性
```

也可以通过 `west build -t menuconfig` 进行可视化配置。

7. SoC 层 + HAL 层 ---》 模块负责人需要注意自己标准库文件是否添加

SoC: `zephyr/soc/nsing/nsing_n32/` (`soc.c`、`Kconfig.defconfig`、`Kconfig.soc` 等)，
提供 `SOC_FAMILY_NSING_N32` 这个驱动依赖的符号

`zephyr/modules/hal_nsing/CMakeLists.txt:29` 库文件 `cmake` 链接目录
`modules\hal\nsing\n32g45x\standard_peripheral\source` 源文件目录

HAL: 驱动 `Kconfig` 里的 `select USE_NSING_I2C` 会拉入 `N32` 的 `HAL` 库 (寄存器定义等)。

这个符号没在 `zephyr/` 里，在外部 HAL 模块里（类似 STM32 的 `USE_STM32_HAL_I2C`），通常在 `modules/` 或独立的 `hal_nsing` 仓库

3 芯片移植

west build -b n32g45xml_stb

1.board 名 → board 目录

```
zephyr/boards/nsing/n32g45xml_stb/board.yaml
-b n32g45xml_stb 在 boards/ 树里查找到 board.yaml:
board:
  name: n32g45xml_stb      # ← 和 -b 参数对上
  vendor: nsing
  socs:
    - name: n32g457        # ← 告诉下一步：这板子用什么芯片**
```

2.board → SoC 目录

```
zephyr/soc/nsing/nsing_n32/soc.yaml
board.yaml 说 SoC 是 n32g457, Zephyr 在 soc/ 树里用 soc.yaml 查这个名字。soc.yaml 定义了层级:

family:
- name: nsing_n32          # 家族
  series:
    - name: n32g45x        # 系列
      socs:
        - name: n32g457    # 芯片名 ← 和 board.yaml 对上
据此定位到目录 soc/nsing/nsing_n32/n32g45x/ (含 soc.c、Kconfig.soc、Kconfig.defconfig)。
```

3.SoC → Kconfig 符号

```
config SOC_SERIES_N32G45X
    bool
    select SOC_FAMILY_NSING_N32      # 最终选中的家族符号

config SOC_N32G457MEL
    bool
    select SOC_SERIES_N32G45X
```

这个 SOC_FAMILY_NSING_N32 就是之前 I2C 驱动 Kconfig 里 depends on SOC_FAMILY_NSING_N32 依赖的符号——它把「这块板 → 这个 SoC → 这个家族」绑定起来，驱动靠它决定自己适不适用。

4.Kconfig 合并 → .config

```
SoC Kconfig.defconfig → Board defconfig → prj.conf → menuconfig
(NUM_IRQS=86 等)      (CONFIG_I2C=y)      (应用配置)      (手动改)
n32g45xml_stb_defconfig 里 CONFIG_I2C=y 就在这里生效。
```

5.设备树合并 + bindings 校验

SoC dtsi (n32g45x.dtsi) → Board dts (n32g45xml_stb.dts) → 应用 overlay
(定义 i2c1 节点, 禁用) (使能 + pinctrl + 速率) (可选)
合并后用 bindings (nsing,n32-i2c.yaml) 校验属性齐全, 生成 devicetree_generated.h
(驱动里的 DT_INST_* 宏就从这里取值)。

build-n32/zephyr/include/generated/zephyr/devicetree_generated.h

设备树里写的	生成的头文件宏	值
reg = <0x40005400 0x400>	..._P_reg_IDX_0 / _IDX_1	1073763328 (=0x40005400) / 1024
clocks = <&rcc N32_CLOCK_I2C1>	..._P_clocks_IDX_0_VAL_bits	1813 (= N32_CLOCK_I2C1)
interrupts = <31 0> (event)	...IRQ_IDX_0_VAL_irq / priority	31 / 0

6.compatible → 驱动

驱动里 #define DT_DRV_COMPAT nsing_n32_i2c, 匹配设备树里的 compatible = "nsing,n32-i2c" (点 → 下划线)。
DT_INST_FOREACH_STATUS_OKAY 遍历所有 status=okay 的实例, 生成设备对象。

7.CONFIG_XXX → 编译哪些源文件

CMakeLists 里的条件编译, 比如 drivers/i2c/CMakeLists.txt:41:
zephyr_library_sources_ifdef(CONFIG_I2C_N32 i2c_n32.c) # .config 里 I2C_N32=y 才编
HAL 同理 (hal_nsing/CMakeLists.txt:42): CONFIG_USE_NSING_I2C → 编 n32g45x_i2c.c。

8. 链接 → zephyr.elf

所有被选中的源文件编成 .o, 链接成 zephyr.elf, 再转 hex/bin 烧录。

4 Zephyr 启动流程

1. 复位—>向量表

复位后 CPU 从 flash 开头的向量表取初始值。

`0x08000000 = 20000FB0` ← 初始 SP (RAM 顶部), 设栈指针 (`0x20000FB0`)

`0x08000004 = 08000A61` ← 复位向量 (PC 跳这里), 跳转到 C 函数 `z_arm_prep_c()`

向量表由链接脚本放在 flash 最开头, SCB->VTOR 指向它。

2. __reset (汇编) → z_arm_prep_c()

__reset 设置好栈后, 进入 C 准备阶段 `prep_c.c`, 做三件事:

```
z_bss_zero();           // ① 清 BSS (按 __bss_start~__bss_end 地址清零)
z_data_copy();          // ② 拷 .data (flash 的 LMA → RAM 的 VMA)
z_arm_platform_init();  // ③ 调 SystemInit() 配时钟
```

段	放哪	启动处理
向量表	flash 开头 0x08000000	无
.text/.rodata	flash	无 (XIP 直接跑)
.data (已初始化变量)	LMA 在 flash, VMA 在 RAM	z_data_copy 从 flash 拷到 RAM
.bss (未初始化变量)	RAM (NOLOAD)	z_bss_zero 清零

.data 的 LMA/VMA 分离由链接脚本的 GROUP_DATA_LINK_IN(RAMABLE_REGION, ROMABLE_REGION) (即 GNU ld 的 AT> 语法) 实现。SystemInit() (system_n32g45x.c) 把时钟从复位默认的 HSI 8MHz 切到 HSE+PLL 144MHz

3. 内核启动

kernel/init.c:539 的 z_cstart() 按顺序:

```
z_sys_init_run_level(INIT_LEVEL_EARLY);    // 早期 init
arch_kernel_init();                        // 架构初始化
LOG_CORE_INIT();                          // 日志核心
z_device_state_init();                     // 设备状态
soc_early_init_hook();                     // SoC 早期钩子 (N32 的 soc.c)
board_early_init_hook();                   // 板级早期钩子
z_sys_init_run_level(INIT_LEVEL_PRE_KERNEL_1); // 基础硬件: GPIO/时钟/中断控制器
z_sys_init_run_level(INIT_LEVEL_PRE_KERNEL_2);
switch_to_main_thread(prepare_multithreading()); // 建主线程并切换
soc_early_init_hook() 在 soc.c 里, 里面有 SystemInit() (更新 CMSIS 的 SystemCoreClock 变量)。
```

4. bg_thread_main()

```
z_sys_init_run_level(INIT_LEVEL_POST_KERNEL); // ★ 设备驱动初始化
soc_late_init_hook();
board_late_init_hook();
z_sys_init_run_level(INIT_LEVEL_APPLICATION); // 应用级 SYS_INIT
z_init_static_threads(); // 静态线程
(void)main(); // ★ 调用你的 main()
```

5. 中断机制（含软件向量表）

Zephyr 支持「动态注册中断」和「ISR 带参数」

表	位置	内容	谁填
硬件向量表 _irq_vector_table[86]	flash	系统异常 + 86 个 _isr_wrapper	编译期生成
软件向量表 _sw_isr_table[]	RAM/特殊 section	{ISR, arg} 对	IRQ_CONNECT 注册

5 git 提交

模块负责人开发，把代码克隆到自己的主机本地

```
git clone git://github.com/derskop/zephyr.git
```

git fetch origin # 把 GitHub 上新建的分支拉到本地

```
git checkout development # 切换并自动关联 origin/development
```